

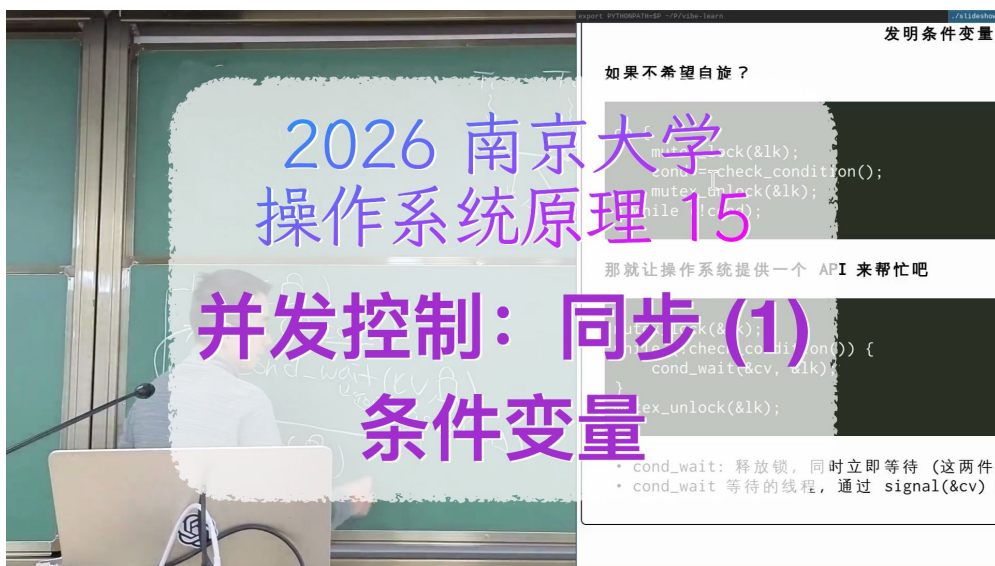
课程笔记

并发控制：条件变量和万能同步方法

南京大学 操作系统原理 2026 春季学期 第 15 讲

lecture-to-notes

2026 年 5 月 7 日



视频作者/频道：绿导师原谅你了（蒋炎岩 jyy）

发布日期：2026 年春季学期

视频时长：1 小时 17 分钟

视频链接：<https://www.bilibili.com/video/BV1t3dQBAEzd>

目录

1 从互斥到同步：并发的”先后”问题	2
2 同步的本质	2
2.1 为什么需要同步：从 wall-clock time 谈起	2
2.2 形象比喻：乐队与约会	3
2.3 计算机播放音乐：用 buffer 把异步包装成”听起来同步”	4
2.4 同步的统一模式：等条件成立	4
2.5 本章小结	5
3 发明条件变量	5
3.1 第一版尝试：自旋等待 (spin loop)	5
3.2 条件变量的三件套 API	5
3.3 条件变量的标准模板	6
3.4 signal vs broadcast：宁可多叫几个	7
3.5 本章小结	7
4 使用条件变量：生产者-消费者与万能同步方法	8
4.1 经典模型：生产者-消费者	8
4.2 万能同步方法	9
4.3 本章小结	10
5 进阶应用：实现有向无环计算图	11
5.1 为什么计算图无处不在？	11
5.2 Makefile 是计算图：make -j 的并行原理	11
5.3 动态规划也是 DAG	12
5.4 本章小结	13
6 总结与延伸	13
6.1 讲师的核心论断	13
6.2 我的结构化提炼	13
6.3 开放问题与下一讲铺垫	13
6.4 拓展阅读	14

1 从互斥到同步：并发的”先后”问题

上一周课程介绍了线程库与互斥锁 lock/unlock。互斥锁解决的是”任意时刻只能一个线程进入临界区”——也就是两段代码 A 和 B 要么先 A 后 B、要么先 B 后 A，但不会交错。而本节课要回答的是另一个并发问题：怎么让 A 一定发生在 B 之前？

互斥不能解决”先后顺序”

互斥锁让多线程对同一段代码”轮流执行”，但顺序仍由调度器决定——谁先抢到锁，谁先进。一个典型场景：主线程 spawn 出 T1, T2, T3，希望”等所有子线程都跑完”再打印 sum。这就是 **thread join** 的需求：

```
1 spawn(T1); spawn(T2); spawn(T3);
2 join();           // 主线程必须等所有子线程结束
3 printf("%d\n", sum); // 才能读出最终结果
```

Listing 1: join 是一种典型的”先后”约束

虽然现在线程库已经把 join 包好了，但理论上 join 自己也应该由**同步原语**来实现——这正是本节课要构建的能力。

2 同步的本质

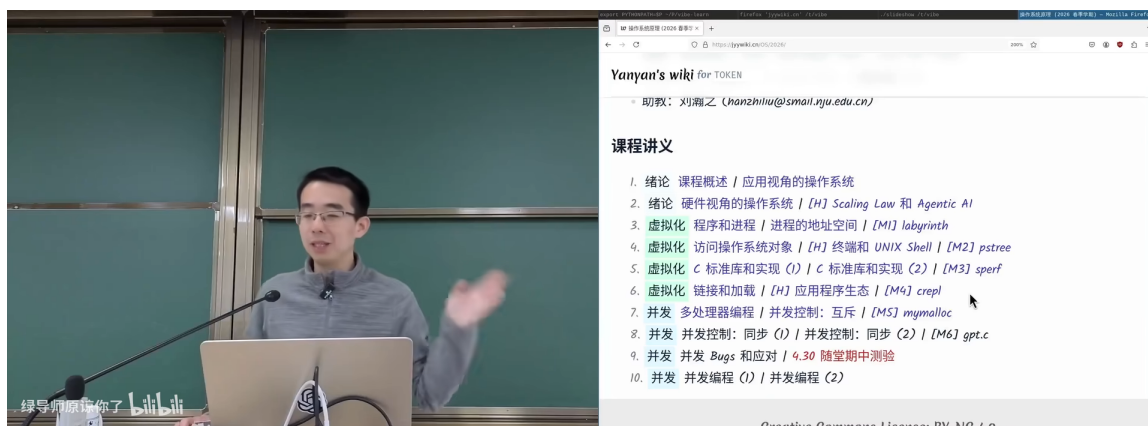


图 1: 并发控制 = 互斥（互不打扰）+ 同步（约定步调）：从只关心”原子性”走到关心”顺序”¹

2.1 为什么需要同步：从 wall-clock time 谈起

异步的字面意思就是各干各的——每个线程是一个独立实体，有自己的私有内存，谁先到谁后到全凭运气。**同步**则是两个或两个以上随时间变化的量在变化过程中保持一定相对关系，听起来抽象，但生活里到处都是：

- 同步电机：用磁场锁定相位
- 同步电路：所有 flip-flop 在时钟上升沿同时锁存
- 上课打铃：约定 wall-clock time 10:10

¹视频画面时间区间：00:00:50–00:01:10。

为什么 wall-clock 同步在计算机里行不通？

时间尺度根本不匹配：

- 人类感官：20–30 ms 以内的差异感受不到
- 计算机：20–30 ms 已经能跑上亿条指令

如果让计算机依靠时间精确同步两个线程，会浪费海量 CPU 周期——空转等时间到。这就是为什么 `sleep(1)` 几乎从来不是合格的同步方案。

2.2 形象比喻：乐队与约会

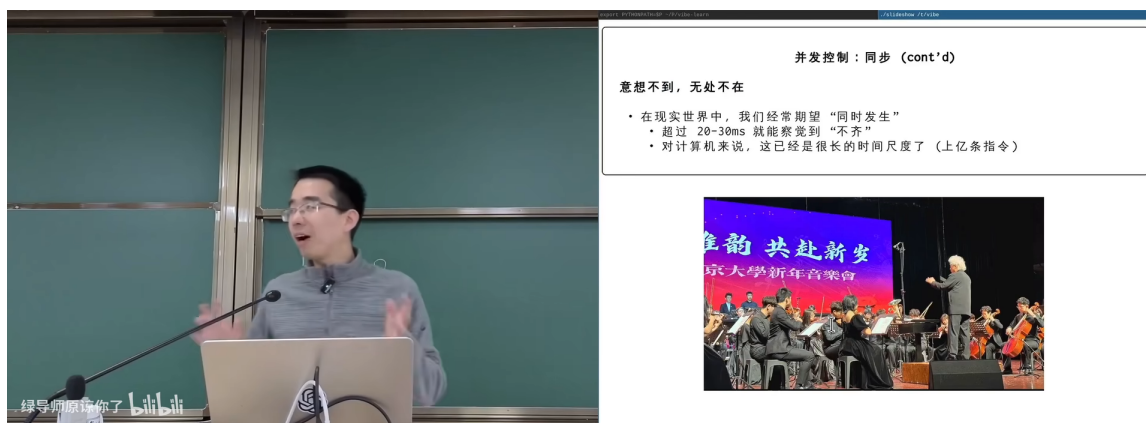


图 2: 乐队的同步：每个乐手都在等待”指挥下一拍”的信号，类似 `while !start: wait2`

两个生活化模型

乐队齐奏（一对多同步）：每个乐手是一个线程，指挥（吕嘉）下一拍的”那个手势”就是同步信号。乐手代码大致是：

```
1 while (!beat_arrived) wait();    // 等下一拍
2 play_note();
```

Listing 2: 乐手线程的伪代码

宿舍约会”不见不散”（多对多同步）：两人约 23:59:59 大活门口，谁先到谁等，直到两人都到齐才”完成同步”。先到的人必须**阻塞等待**，等到对方到达时才继续。这正是所有同步原语的物理本质——**握手**。

²视频画面时间区间：00:07:00–00:07:30。讲师以新年音乐会乐队为例：人类感觉得到 20–30 ms 不同步，计算机要做到这点其实很容易。

2.3 计算机播放音乐：用 buffer 把异步包装成”听起来同步”

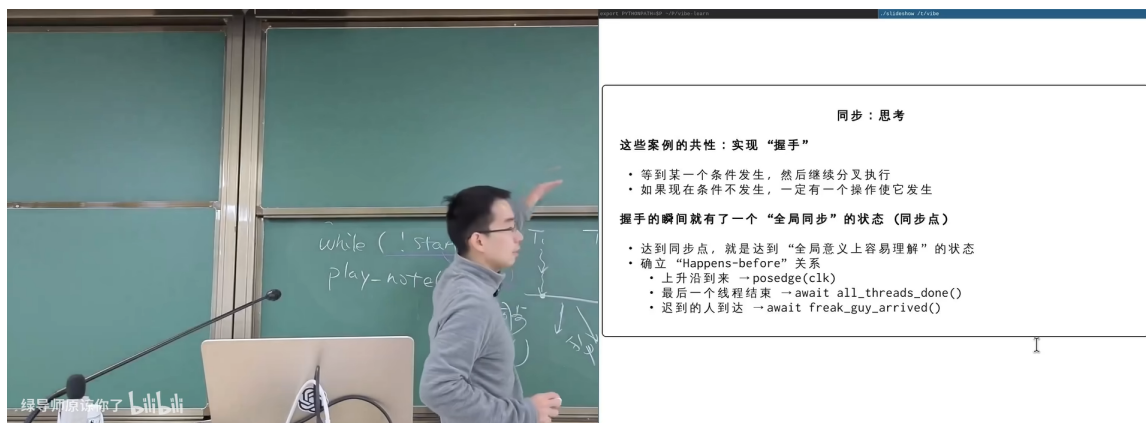


图 3：音频播放：硬件 buffer + 软件提前填充——计算机能”无缝播放”的真正机制³

电脑放音乐为何”从来不卡”？关键不是 CPU 一直在算，而是：

1. 硬件设备维护一个固定大小的环形 buffer（比如 20 ms 的样本）
2. 软件每次塞 1 块进去，硬件按时钟频率连续读出
3. 只要软件能在下一个 buffer 用完前补上新数据，听起来就是连续的

人耳的”宽容窗口”约 20–30 ms，硬件 buffer 把这段宽容期吃掉了，软件才得以异步喂数据。
同步 = 协议 + 等待，而不是物理时间一致——这是本节课最重要的思维转换。

2.4 同步的统一模式：等条件成立

把上面所有例子抽象出来，每一个同步问题都可以写成同一个模板：

```
1 while (!sync_condition_met()) {  
2     wait();    // 阻塞，等待条件被改变  
3 }  
4 proceed();    // 条件成立后继续
```

Listing 3: 同步问题的统一抽象

- 乐手： `sync_condition` = ”下一拍来了”
- 不见不散： `sync_condition` = ”两人都到齐”
- 主线程 join： `sync_condition` = ”所有子线程已结束”
- 同步电路 flip-flop： `sync_condition` = ”时钟边沿到达”

剩下的问题就是：怎么实现 `wait()` 才能既正确，又不浪费 CPU？

³视频画面时间区间：00:08:30–00:09:50。

2.5 本章小结

- 互斥解决”互不打扰”，同步解决”先后顺序”
- Wall-clock 同步在 CPU 时间尺度下不可行
- 所有同步问题都是 `while (!cond) wait()` 这一个模式
- 关键挑战：`wait()` 怎么实现才不浪费 CPU

3 发明条件变量

3.1 第一版尝试：自旋等待 (spin loop)

最朴素的实现就是忙等：

```
1 mutex_lock(&m);
2 while (!condition) {           // 条件不满足
3     mutex_unlock(&m);          // 让别人有机会改 condition
4     /* spin */
5     mutex_lock(&m);
6 }
7 do_critical_section();
8 mutex_unlock(&m);
```

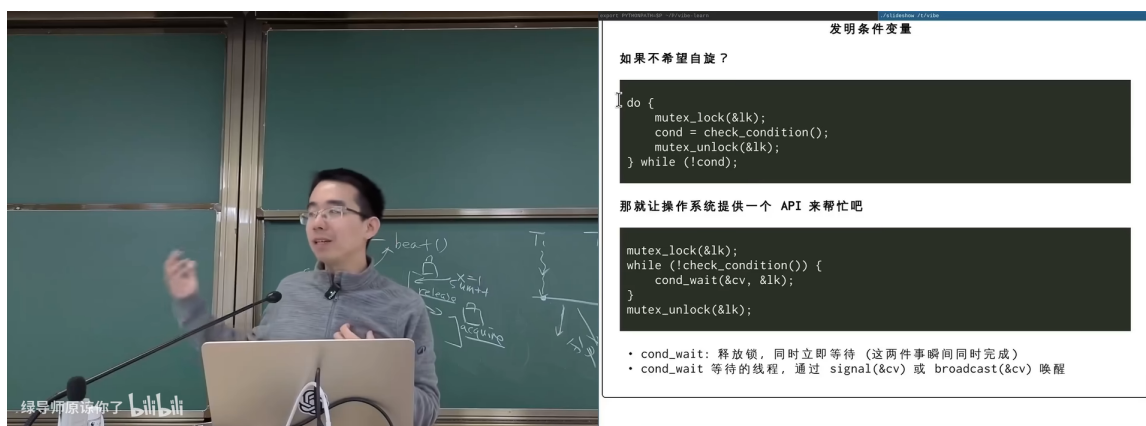
Listing 4: 自旋等待——能跑但浪费 CPU

自旋的两个问题

1. **浪费 CPU**：等待者占满一整个核反复 acquire / release / 检查
2. **窗口竞争**：刚释放锁的瞬间到下一次 acquire 之间，可能错过 condition 已经被满足的瞬间——但这一版还是正确的，只是非常吵

我们需要一个机制让线程”睡过去”，等条件可能成立时再被叫醒。

3.2 条件变量的三件套 API



发明条件变量

如果不希望自旋？

```
do {
    mutex_lock(&l);
    cond = check_condition();
    mutex_unlock(&l);
} while (!cond);
```

那就让操作系统提供一个 API 来帮忙吧

```
mutex_lock(&l);
while (!check_condition()) {
    cond_wait(&cv, &l);
}
mutex_unlock(&l);
```

- `cond_wait`: 释放锁，同时立即等待（这两件事瞬间同时完成）
- `cond_wait` 等待的线程，通过 `signal(&cv)` 或 `broadcast(&cv)` 唤醒

图 4: 从 spin loop 到 `cond_wait`：思路就是把”忙等”换成”睡眠 + 被叫醒”⁴

操作系统/库提供 **条件变量** (condition variable, `cond_t`), 配合 `mutex` 使用。三个 API:

- `cond_wait(&cv, &mutex)`: **原子地**释放 `mutex` + 进入睡眠队列
- `cond_signal(&cv)`: 唤醒一个睡在该 `cv` 上的线程
- `cond_broadcast(&cv)`: 唤醒全部睡在该 `cv` 上的线程

`cond_wait` 的原子性是关键

`cond_wait` 必须保证下面两步**不能被打断**:

1. 释放 `mutex`
2. 把当前线程放入 `cv` 的睡眠队列

如果不是原子的, 就可能出现: 刚释放锁 → 别人 `signal` 来 → 但我还没进队列, 错过通知 → **永久睡死**。这是初学者最容易掉进去的坑。库实现时一定会用某种内核机制 (比如 `futex`) 保证这两步原子。

被唤醒后 `cond_wait` 会自动重新获取 `mutex`, 再返回。所以从调用者看, 整段代码就好像被同一把锁连续保护着。

3.3 条件变量的标准模板

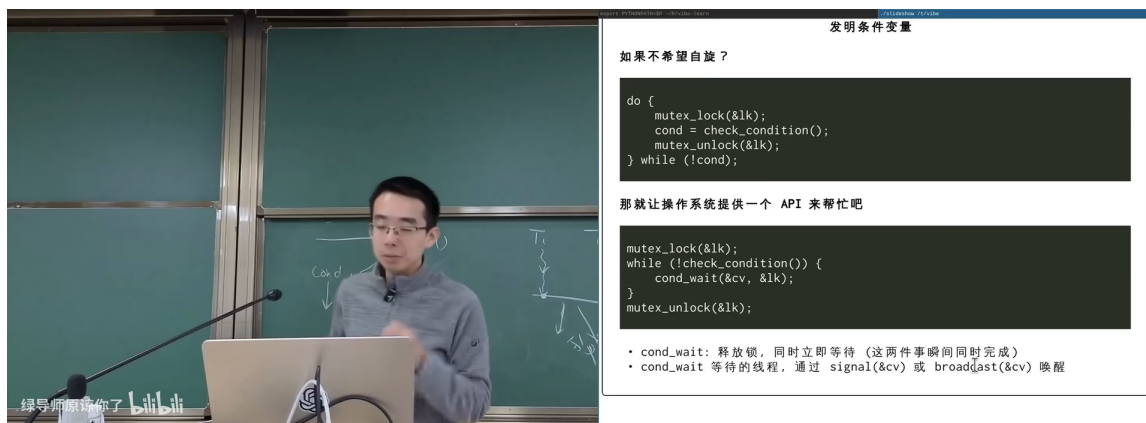


图 5: 条件变量使用模板——讲师称”期末考试 20 分的代码”⁵

```
1 mutex_lock(&m);
2 while (!condition) {           // 注意是 while 不是 if
3     cond_wait(&cv, &m);        // 原子释放锁 + 睡眠
4 }
5 /* 这里 condition 一定成立, 且 mutex 在手 */
6 do_critical_work();
7 mutex_unlock(&m);
```

Listing 5: 条件变量的”期末考试”模板 (jyy 称值 20 分)

⁴视频画面时间区间: 00:25:00–00:25:30。

⁵视频画面时间区间: 00:30:00–00:30:30。

为什么是 while, 不是 if ?

Mesa 语义下唤醒不等于条件成立: 被 signal 叫醒只是”也许条件可能成立了, 自己再去检查”。可能:

1. 假唤醒 (spurious wakeup, POSIX 允许)
2. 多线程被一起唤醒, 只有一个能让条件继续成立
3. 条件刚被改回去

写 if 在所有这些情况都可能让 condition 不成立时就往下走, 导致正确性事故。while 让线程被唤醒后重新检查, 确认成立才继续。这是**无脑模板**: 用 while 永远不会错。

3.4 signal vs broadcast: 宁可多叫几个

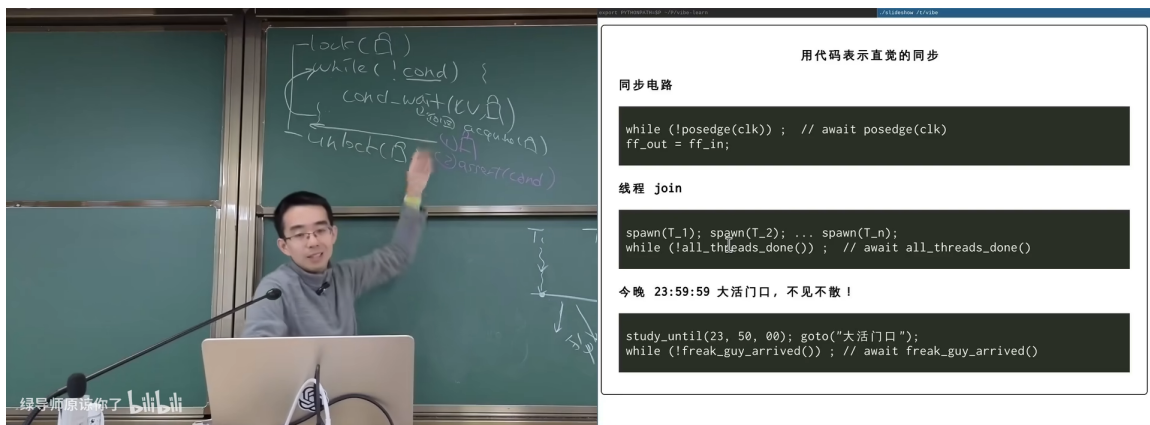


图 6: signal 唤醒一个、broadcast 唤醒全部——broadcast 略费 CPU 但永远正确⁶

- `cond_signal`: 性能更好 (只叫一个), 但要求你**确信**被叫醒的那个线程的条件一定成立
- `cond_broadcast`: 把所有等待者都叫起来, 每个自己检查条件——多绕几圈但**永不出错**

讲师的”无脑建议”

”只要你发现任何一个线程修改了状态、使得别人的同步条件可能达成——都用 *broadcast*。”理由: 用 *signal* 时你必须证明”被叫醒的那个一定能往下跑”, 但实际程序的同步条件常常彼此重叠 (比如同一个 `cv` 上有读者和写者), *signal* 给错了人就会导致饥饿或死锁。*broadcast* + *while* 是**鲁棒组合**: 略损一点性能, 换永远正确。

3.5 本章小结

- 条件变量 = 让”`wait()`”真正睡过去而不忙等
- `cond_wait` 必须原子地完成”释放锁 + 入队睡眠”两步
- 唤醒后 `cond_wait` 会自动重新获取锁

⁶视频画面时间区间: 00:35:00-00:35:30。

- 模板: mutex_lock → while (!cond) cond_wait → 临界区 → mutex_unlock
- while 不是 if; broadcast 优于 signal

4 使用条件变量：生产者-消费者与万能同步方法

4.1 经典模型：生产者-消费者

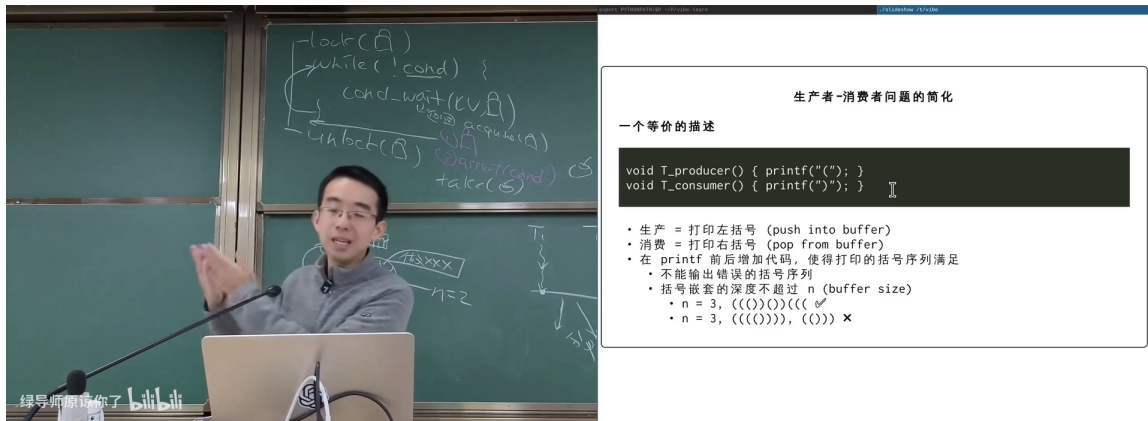


图 7: 生产者-消费者: 共享一个有界缓冲区, producer 往里放、consumer 往外取⁷

生产者-消费者: scheduler-worker 模式的经典化

设有一张桌子 (容量 N), 上面放苹果 (带任务字条):

- **Producer** 不断生产苹果放桌上; 桌满则等
- **Consumer** 不断从桌上取苹果做事; 桌空则等

讲师强调: "只要掌握这一个模型, 你基本上就可以解决 99% 的实际并发问题。" 本质上这就是 master-slave / scheduler-worker 模式——它换了个政治正确的名字, 但核心是用一个共享数据结构 (队列/桌子) + 同步原语解耦上下游。

```

1  /* 共享状态 */
2  mutex_t m;
3  cond_t  not_full, not_empty;
4  int     n = 0;          /* 当前桌上苹果数 */
5
6  void produce() {
7      mutex_lock(&m);
8      while (n == N)      /* 桌满 */
9          cond_wait(&not_full, &m);
10     put_apple();
11     n++;
12     cond_broadcast(&not_empty);
13     mutex_unlock(&m);
14 }

```

⁷ 视频画面时间区间: 00:42:00-00:42:30。

```

15
16 void consume() {
17     mutex_lock(&m);
18     while (n == 0)          /* 桌空 */
19         cond_wait(&not_empty, &m);
20     take_apple();
21     n--;
22     cond_broadcast(&not_full);
23     mutex_unlock(&m);
24 }

```

Listing 6: 生产者-消费者的标准条件变量实现

为什么要两个 cv ?

桌子有两个边界条件：**满**（producer 等）和 **空**（consumer 等）。

- 如果只用一个 cv: consumer 取走一个苹果应该叫醒等待“非满”的 producer, 但 signal 可能叫醒了另一个 consumer, 对方却找不到苹果——程序卡死或唤醒风暴
- 拆成两个 cv: not_full 给 producer 等、not_empty 给 consumer 等, 互不打扰

两个等待条件 → 两个 cv, 是写并发程序时反复出现的设计直觉。

4.2 万能同步方法

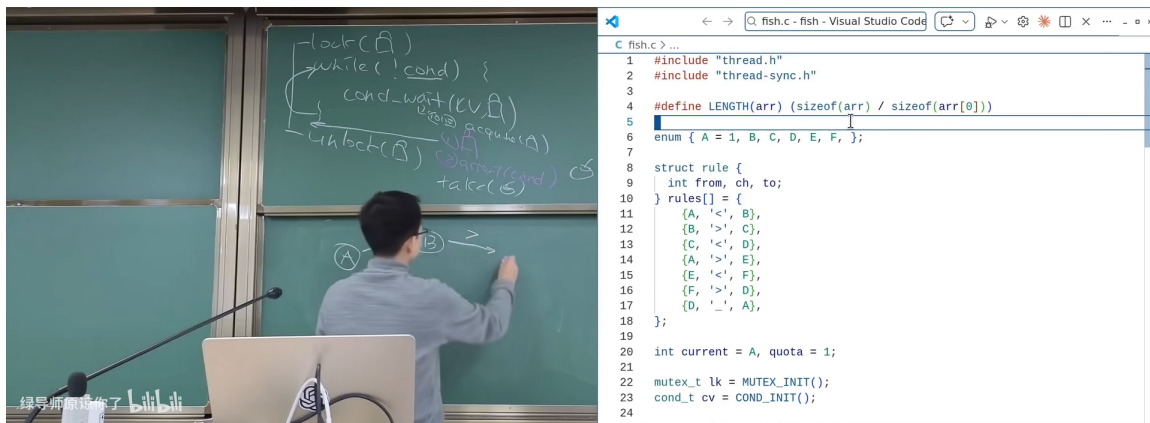


图 8: 万能同步方法: 任何同步问题都可以套到 producer-consumer 模板⁸

⁸视频画面时间区间: 00:54:00-00:54:30。讲师明示: 把任何“等条件成立”的问题都翻成 producer-consumer。

把任何同步问题翻成 producer-consumer

万能同步方法的核心思路：

1. 识别 **sync condition**：哪两件事必须有先后？
2. 把它建模成”消费一个事件”：后做的那件事就是 consumer，等待事件
3. 先做的那件事 **produce 这个事件**：完成后 **cond_broadcast**
4. 用一个共享变量记录事件状态（标志位 / 计数器 / 队列），保护它的 **mutex** 和 **cv** 配套使用

例：**thread join** 就是主线程 **consume counter == 0**、子线程 **produce counter--**：

```
1 int      n_running = N;
2 mutex_t m;
3 cond_t  done;
4
5 void child_thread() {
6     do_work();
7     mutex_lock(&m);
8     n_running--;
9     if (n_running == 0)
10         cond_broadcast(&done);
11     mutex_unlock(&m);
12 }
13
14 void join_all() {
15     mutex_lock(&m);
16     while (n_running > 0)
17         cond_wait(&done, &m);
18     mutex_unlock(&m);
19 }
```

Listing 7: join 的条件变量实现（万能同步方法应用）

4.3 本章小结

- 生产者-消费者覆盖 99% 实际并发问题
- 两个等待条件就用两个 **cv**，每个搭配自己的”while !cond”
- 万能同步方法：把同步问题翻成”消费事件”——先做的人 **produce**、后做的人 **wait**
- **join** 不是魔法，是 **producer-consumer** 的最简实例

5 进阶应用：实现有向无环计算图

5.1 为什么计算图无处不在？

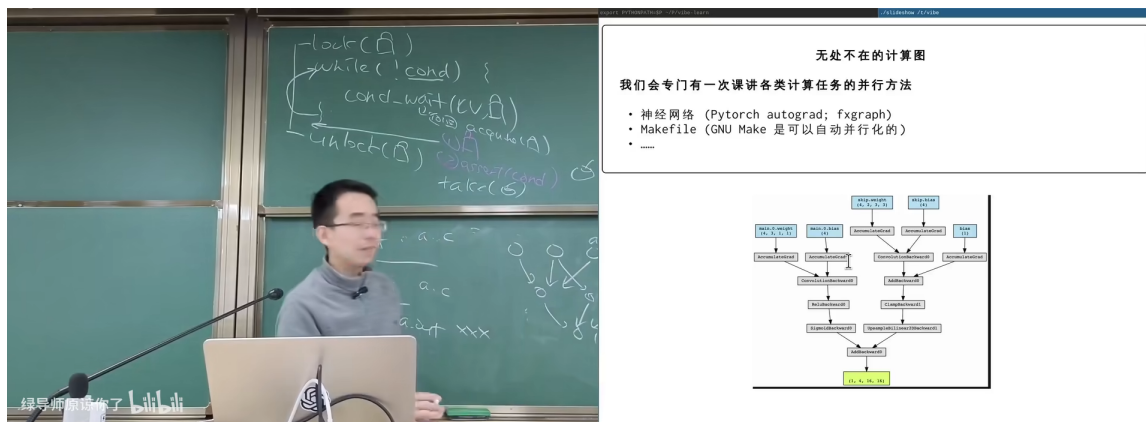


图 9: PyTorch fx graph: 神经网络前向计算就是一个有向无环图⁹

你以为是孤立例子，其实都是 DAG

日常工程中至少这些都是有向无环计算图 (DAG):

- 神经网络: PyTorch fx graph、TensorFlow Graph
- Makefile: 每条 recipe 是一个节点，依赖是有向边
- 动态规划: 状态转移图 (如最长公共子序列)
- Spark / Airflow 工作流: 任务编排

它们共同的执行需求: 节点 v 必须等所有前驱节点完成后才能开始。互不依赖的节点可以并行。这就是 DAG 调度问题，也是同步问题。

5.2 Makefile 是计算图: make -j 的并行原理

make 命令会自动并行化构建图——-j16 表示同时跑 16 个 worker:

```
1 a.out: a.o b.o
2     gcc a.o b.o -o a.out
3 a.o: a.c
4     gcc -c a.c
5 b.o: b.c shared.h
6     gcc -c b.c
```

Listing 8: Makefile 的依赖关系本质上是 DAG

a.o 和 b.o 不互相依赖 → 可以并行编译; a.out 必须等两者都编译完。make 内部就是用类似条件变量的机制等待依赖项完成的——也就是每个节点完工时 broadcast，每个等待者 while !ready: cond_wait。

⁹ 视频画面时间区间: 01:00:30-01:01:30。

5.3 动态规划也是 DAG

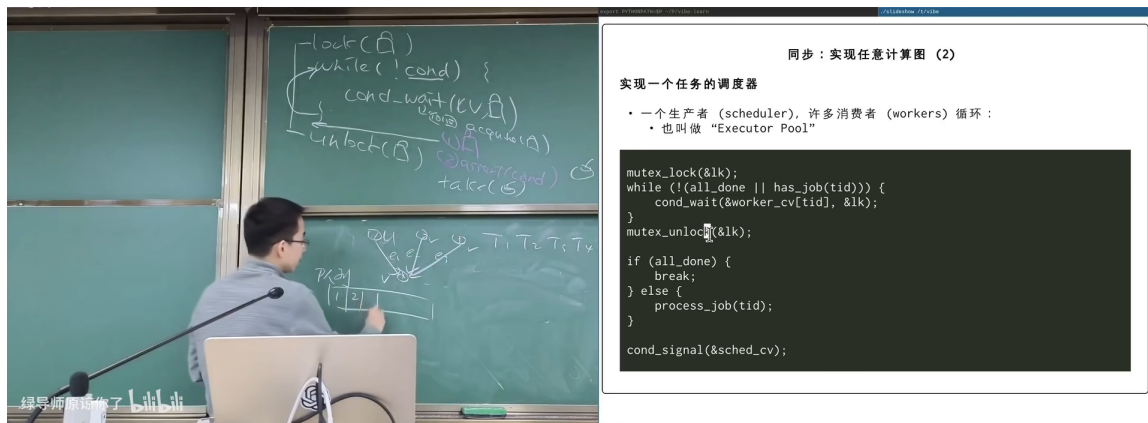


图 10: 最长公共子序列 LCS 的状态转移图: 每个 $dp[i][j]$ 依赖左、上、左上三个邻居¹⁰

把任意 DAG 调度落到条件变量

对每个节点 v 维护:

- $n_pending_deps$: 还没就绪的前驱数
- $ready$: 本节点是否已完成

调度逻辑:

```
1 void run_node(Node *v) {
2     /* 等所有前驱完成 */
3     mutex_lock(&v->m);
4     while (v->n_pending_deps > 0)
5         cond_wait(&v->ready_cv, &v->m);
6     mutex_unlock(&v->m);
7
8     compute(v);
9
10    /* 通知后继 */
11    for (Node *succ : v->successors) {
12        mutex_lock(&succ->m);
13        succ->n_pending_deps--;
14        if (succ->n_pending_deps == 0)
15            cond_broadcast(&succ->ready_cv);
16        mutex_unlock(&succ->m);
17    }
18 }
```

Listing 9: DAG 节点的同步逻辑 (用条件变量实现)

实际工程中 (PyTorch / Spark) 会用更高效的调度池替代 cv , 但逻辑骨架完全一样: 节点等”前驱全部就绪”, 完工后广播。

¹⁰ 视频画面时间区间: 01:05:00–01:05:30。

5.4 本章小结

- 计算图 = 神经网络 + Makefile + 动态规划 + 工作流的统一抽象
- DAG 调度等价于：每个节点等”前驱就绪”、完工后广播
- 用条件变量可以直接写出最朴素正确的并行执行器
- 万能同步方法在 DAG 上的具象化：每条依赖边就是一次 producer-consumer

6 总结与延伸

6.1 讲师的核心论断

一节课讲完后你应该相信

- 互斥 → 解决”谁也别打扰谁”
- 同步 → 解决”先后顺序”
- 条件变量 = 让等待不耗 CPU 的 `wait()`
- 生产者-消费者 + 万能同步方法 = 99% 现实并发问题的统一解法

”你只要学会了同步，你基本上 99% 的现实世界中复杂的并行计算问题，都可以在这里找到答案。”

6.2 我的结构化提炼

	层级	工具	解决的问题
三层抽象	原语层	<code>mutex, cond</code>	互斥 + 等条件成立
	模式层	生产者-消费者	任意”事件驱动”同步
	应用层	DAG 调度	计算图 / 工作流的并行执行

条件变量心法

1. 永远 `while` 不 `if` (防假唤醒、防多人争抢)
2. 永远 `broadcast` 不 `signal` (牺牲少量 CPU 换鲁棒性)
3. 每个不同等待条件配独立 `cv` (防止误唤醒风暴)
4. `cond_wait` 前先持锁、`wait` 内会原子释放再重抢 (不要自己手动 `unlock-wait`)

6.3 开放问题与下一讲铺垫

- 条件变量虽然是表达力最强的同步原语,但代码量大、容易出错。下一讲将引入信号量(`semaphore`)——它在某些场景(如限流、读者-写者)写起来比 `cv` 更简洁
- 同步条件在多个变量上耦合时,怎么避免死锁? 这是第 17 讲”并发 bugs 与应对”的主题
- 工程实践中 `std::condition_variable`、Java `Object.wait/notify`、Go `channel` 都是条件变量的不同包装;学会本节后可以触类旁通

6.4 拓展阅读

- *Operating Systems: Three Easy Pieces* (OSTEP) 第 30 章 Condition Variables
- POSIX `pthread_cond_*` 手册
- Lampson & Redell, "Experience with Processes and Monitors in Mesa," *CACM* 1980 (Mesa 语义的来源)